BANK ACCOUNT AND MONEY TRANSFERS

CASE STUDY: BANKING APPLICATION

Subject: Advanced Object-Oriented Programming

Submitted By:

Name: Vaibhava Sree

Roll Number: 2520030398

Section: 4

Submitted To:

Faculty Name: Dr. Ramya Krishna Dhulipalla

Department: CSE



Academic Year: 2026–2027

1. AIM

To design and implement a banking application using Java and object-oriented programming concepts.

The application supports bank account creation, deposits, withdrawals, money transfers, account freezing, unfreezing, account closing, overdraft limits, and transaction history management.

2. INTRODUCTION

A banking application is a software system used to manage customers' bank accounts and financial transactions.

It allows customers to deposit money, withdraw money, transfer funds, and check their account balances.

While developing a banking application, several problems may occur, such as invalid transactions, incorrect account balances, interrupted money transfers, and unauthorized account operations.

To solve these problems, we use object-oriented programming concepts such as encapsulation, abstraction, classes, objects, and immutability.

In this case study, we develop a simple banking application using Java. The application maintains account information, validates transactions, and stores transaction history.

3. PROBLEM STATEMENT

A retail banking application must support the following operations:

1. Create bank accounts.
2. Deposit money into accounts.
3. Withdraw money from accounts.
4. Transfer money between accounts.
5. Freeze and unfreeze accounts.
6. Close bank accounts.
7. Maintain transaction history.
8. Apply overdraft limits.

The application must prevent invalid operations and maintain consistency between account balances and transaction records.

4. DESIGN PROBLEMS IDENTIFIED

The following problems may occur while developing the banking application:

1. Invalid Transaction Amount: The system may accept zero or negative deposit, withdrawal, or transfer amounts.
2. Overdraft Limit Violation: Customers may withdraw more money than their permitted overdraft limit.
3. Frozen Account Transactions: Frozen accounts may incorrectly allow withdrawals or other restricted operations.
4. Closed Account Transactions: Closed accounts may continue accepting financial transactions.
5. Interrupted Fund Transfer: If a transfer fails after debiting the source account but before crediting the destination account, money may be lost.
6. Incorrect Account Balance: Account balances may not match the actual transactions performed.
7. Ledger Inconsistency: Transaction history may not correctly reflect deposits, withdrawals, and transfers.
8. Duplicate Transactions: The same transaction may be processed more than once.
9. Currency Mismatch: Transactions involving different currencies may produce incorrect results.
10. Transaction Record Modification: If ledger entries are mutable, transaction history may be changed incorrectly.
11. Account Closure with Remaining Balance: An account may be closed even when money remains in it.
12. Concurrent Transaction Problems: Multiple transactions performed simultaneously may cause incorrect balance updates.
13. Transfer Failure Recovery: The system may fail to restore the source account balance when a transfer is interrupted.
14. Large Transaction History: Storing too many transaction records may reduce application performance.

15. Lack of Transaction Security: Unauthorized users may access or modify account information if proper security controls are not implemented.

5. Q1: DOMAIN INVARIANTS

What is an Invariant?

An invariant is a rule or condition that must always remain true in a system.

In simple words, an invariant is a rule that should never be broken.

Domain Invariants in a Banking Application

1. Positive Transaction Amount

Rule: Every deposit, withdrawal, and transfer amount must be greater than zero.

Example: Depositing ₹5,000 is valid, but depositing ₹0 is invalid.

Importance: Prevents meaningless or invalid transactions.

Consequence of violation: Zero or negative transactions may produce incorrect account balances.

2. Overdraft Limit

Rule: An account balance must never go below its permitted overdraft limit.

Example: If the overdraft limit is ₹2,000, the balance cannot be less than -₹2,000.

Importance: Prevents customers from withdrawing more than the allowed amount.

Consequence of violation: The bank may allow unauthorized withdrawals.

3. Account Status Validation

Rule: Frozen and closed accounts must not allow normal financial transactions.

Example: A frozen account should not allow withdrawals.

Importance: Protects restricted and closed accounts.

Consequence of violation: Unauthorized transactions may occur.

4. Transfer Consistency

Rule: When money is transferred, the same amount must be deducted from one account and added to another.

Example: If ₹3,000 is deducted from Account A, ₹3,000 must be credited to Account B.

Importance: Ensures that money is not lost or duplicated during transfers.

Consequence of violation: One account may lose money without the other receiving it.

5. Ledger Consistency

Rule: Every successful transaction must be recorded in the transaction history.

Example: If ₹5,000 is deposited, the ledger must contain a deposit record of ₹5,000.

Importance: Maintains accurate financial history.

Consequence of violation: Account balances and transaction records may disagree.

6. Currency Consistency

Rule: Money operations must use the same currency unless currency conversion is supported.

Example: INR cannot be directly added to USD.

Importance: Prevents incorrect monetary calculations.

Consequence of violation: Financial calculations may become inaccurate.

7. Account Closure

Rule: An account should be closed only when its balance is zero.

Example: An account with ₹500 cannot be closed.

Importance: Prevents closing accounts with unsettled balances.

Consequence of violation: Remaining funds may become difficult to manage.

8. Unique Transaction ID

Rule: Every transaction must have a different transaction ID.

Example: Two different deposits must have separate transaction IDs.

Importance: Helps identify and track individual transactions.

Consequence of violation: Transactions may become difficult to distinguish.

Conclusion: Domain invariants help prevent invalid operations and maintain the correctness, security, and consistency of the banking application.

6. Q2: ENTITIES AND VALUE OBJECTS

What is an Entity?

An entity is an object that has a unique identity and can change its properties over time.

Example: A Bank Account has a unique account number. Its balance may change, but its identity remains the same.

What is a Value Object?

A value object is an object that represents a value rather than a unique identity. It is generally immutable and compared using its values.

Example: Money represents an amount and currency. INR 500 is identified by its amount and currency.

Classification in the Banking Application

1. Bank Account – Entity

A bank account has a unique account number, balance, and status. Its balance can change, but its identity remains the same.

2. Account Number – Value Object

It represents the number used to identify a bank account. Its value is important rather than its independent lifecycle.

3. Money – Value Object

It represents an amount and currency. Its value depends on these properties.

4. Ledger Entry – Entity

It represents an individual transaction and has a unique transaction ID.

5. Overdraft Limit – Value Object

It represents the maximum amount an account is allowed to go below zero.

6. Fund Transfer – Entity

It represents a money transfer and has a unique transfer ID and status.

Difference Between Entity and Value Object

Entity	Value Object
Has a unique identity	Does not need an independent identity
Can change over time	Generally immutable
Compared using identity	Compared using values
Example: Bank Account	Example: Money

Conclusion: Entities and value objects help organize the application properly and make the code easier to understand and maintain.

7. Q3: SELF-ENFORCING OBJECTS AND JAVA DESIGN

What is a Self-Enforcing Object?

A self-enforcing object is an object that protects its own data and prevents invalid operations.

In simple words, an object checks the rules before allowing any operation.

Example: If a customer tries to withdraw money from a frozen account, the BankAccount object checks the account status and rejects the withdrawal.

7.1 Money Class

The Money class represents monetary values.

- Amount stored in paise.
- Currency.
- Addition operation.
- Subtraction operation.
- Positive amount checking.
- Currency compatibility checking.

The amount is stored in paise to avoid common floating-point precision problems.

7.2 BankAccount Class

The BankAccount class represents a bank account.

Attributes:

- Account number
- Balance
- Overdraft limit
- Account status
- Transaction ledger

Methods:

- deposit()
- withdraw()
- freeze()
- unfreeze()
- close()
- getBalance()

The class validates every operation before modifying the account.

7.3 AccountStatus Enum

The AccountStatus enum defines three possible account states: ACTIVE, FROZEN, and CLOSED. This ensures that account status is selected from predefined values.

7.4 LedgerEntry Class

The LedgerEntry class stores transaction details: Transaction ID, Transaction type, Transaction amount, and Timestamp. Its fields are final, so the transaction details cannot be reassigned after the object is created.

7.5 How Does the Application Prevent Invalid Operations?

1. Invalid amounts are rejected.
2. Frozen accounts cannot perform normal financial transactions.

3. Closed accounts cannot perform transactions.
4. Withdrawals cannot exceed the overdraft limit.
5. Currency mismatches are rejected.
6. Account data is protected using private variables.

7.6 What is Encapsulation?

Encapsulation is an object-oriented programming concept in which data is protected by restricting direct access to it.

In this application, account fields are declared private. Public methods are provided to perform operations such as deposits and withdrawals.

7.7 What is Immutability?

Immutability is a property of an object whose data cannot be modified after it has been created.

Example: Once a ledger entry records a deposit of ₹5,000, its amount cannot be directly changed to ₹2,000.

If a correction is needed, a new reversal or adjustment entry should be created.

Importance: Protects data from unwanted changes; maintains transaction history accuracy; prevents accidental modifications; improves data reliability.

Conclusion: Self-enforcing objects protect data, prevent invalid operations, and maintain the correctness of the application.

8. Q4: MONEY TRANSFER AND AGGREGATE CONSISTENCY

What is Money Transfer?

Money transfer means moving money from one bank account to another.

What is Aggregate Consistency?

Aggregate consistency means maintaining the rules and correctness of related objects during an operation.

8.1 Interrupted Transfer Problem

Suppose Account A transfers ₹5,000 to Account B. The following operations must happen:

1. Check whether both accounts are active.
2. Check whether the transfer amount is positive.
3. Check whether Account A has sufficient balance.
4. Deduct ₹5,000 from Account A.
5. Add ₹5,000 to Account B.
6. Record the transaction.
7. Mark the transfer as completed.

Suppose the system crashes after deducting ₹5,000 from Account A but before adding it to Account B.

The following problems may occur: Account A loses ₹5,000; Account B does not receive ₹5,000; account balances become inconsistent; transaction history may become incorrect.

8.2 Violated Invariants

An interrupted transfer may violate: transfer consistency, ledger consistency, account balance consistency, and transaction completion rules.

8.3 How Can Transfer Failure Be Prevented?

A proper banking system should use database transactions. It should follow these steps:

1. Start a database transaction.
2. Validate both accounts.
3. Debit the source account.
4. Credit the destination account.
5. Record the transaction.
6. Commit all changes together.

If an operation fails before the transaction is committed, the database should roll back the changes.

8.4 What are ACID Properties?

1. Atomicity

Either all operations succeed or none are committed.

2. Consistency

All database rules remain valid after a transaction.

3. Isolation

Multiple transactions should not interfere incorrectly with each other.

4. Durability

Committed changes remain saved even after a system failure.

8.5 Failure Handling

The current Java application demonstrates a basic compensating refund attempt if destination crediting fails after the source debit.

However, this in-memory implementation does not guarantee recovery from a sudden application or system crash.

A production banking application should use database transactions, durable transaction records, unique transaction identifiers, and recovery mechanisms.

8.6 Cross-Bank Transfers

When money is transferred between different banks, the system may require:

- Unique transaction identifiers.
- Transaction status tracking.
- Confirmation from both banks.
- Retry mechanisms.
- Reconciliation.
- Reversal or compensation mechanisms.
- Distributed transaction or Saga-based processing.

Conclusion: Transfer consistency prevents money loss, duplicate transfers, and incorrect account balances.

9. Q5: LEDGER MANAGEMENT AND DESIGN EVALUATION

What is a Ledger?

A ledger is a record of all financial transactions performed on a bank account.

In simple words, it is the transaction history of an account.

9.1 Information Stored in a Ledger

1. Transaction ID
2. Transaction type
3. Transaction amount
4. Timestamp
5. Account number
6. Transaction status
7. Transaction description

The current Java implementation stores the transaction ID, transaction type, amount, and timestamp in each LedgerEntry.

9.2 What are Immutable Ledger Entries?

An immutable ledger entry is a transaction record whose data cannot be changed after it is created.

Example: Once a deposit of ₹5,000 is recorded, its amount should not be directly changed to ₹3,000.

If a correction is required, a separate reversal or adjustment entry should be created.

9.3 How is Ledger Consistency Maintained?

1. Every successful transaction must be recorded.
2. Account balances and ledger records must be updated together.
3. Transaction entries should not be modified or deleted.
4. Transaction IDs should be unique.
5. Direct modification of account balances should not be allowed.

In a production application, balance updates and ledger writes should be committed together in a database transaction.

9.4 Challenges with Large Transaction History

1. Increased storage requirements.
2. Slower transaction history retrieval.
3. Increased processing time.
4. Backup and recovery difficulties.
5. Security and privacy concerns.

9.5 Solutions

- Use database indexing.
- Use pagination.
- Archive old transaction records.
- Maintain regular backups.
- Apply proper security controls.

Conclusion: Ledger management helps maintain accurate transaction history, track financial activities, and verify account balances.

10. JAVA CLASS DESCRIPTIONS

The application contains six Java files.

10.1 Money.java

The Money class represents monetary values. Main features: stores amount in paise; stores currency; adds and subtracts money; checks positive amounts; prevents currency mismatch.

10.2 AccountStatus.java

The AccountStatus enum defines three possible account states: ACTIVE, FROZEN, and CLOSED.

10.3 LedgerEntry.java

The LedgerEntry class stores transaction details and timestamps. Its fields are final, making transaction details immutable after initialization.

10.4 BankAccount.java

The BankAccount class manages individual bank accounts. Main features: creates accounts; maintains balances; deposits money; withdraws money; checks overdraft limits; freezes and unfreezes accounts; closes accounts; maintains transaction history.

10.5 FundTransfer.java

The FundTransfer class manages money transfers between two bank accounts. Main features: generates a unique transfer ID; stores source and destination accounts; validates transfer amounts; checks account status and currency; performs debit and credit operations; maintains transfer status.

10.6 Main.java

The Main class is the entry point of the application. It creates bank accounts and performs deposit, withdrawal, transfer, freeze, unfreeze, overdraft, invalid amount, and account closure tests.

11. OBJECT-ORIENTED PROGRAMMING CONCEPTS USED

OOP concept	Meaning	Application
Encapsulation	Protecting data by restricting direct access	Private account fields
Abstraction	Hiding internal details and showing necessary operations	Deposit and withdrawal methods
Classes and Objects	Creating objects from classes	Money, BankAccount, LedgerEntry
Inheritance	Acquiring properties of another class	Not required in this implementation
Polymorphism	One interface or method having different behaviours	Not explicitly used in this implementation
Enum	Defining a fixed set of values	AccountStatus
Immutability	Preventing changes after object creation	Money and LedgerEntry design

The application mainly uses encapsulation, abstraction, classes, objects, and immutable value objects.

12. APPLICATION WORKFLOW

1. Start the application.
2. Create Money objects.
3. Create bank accounts.

4. Display initial account details.
5. Perform a deposit.
6. Perform a withdrawal.
7. Transfer money between accounts.
8. Display updated balances.
9. Freeze an account.
10. Test withdrawal from a frozen account.
11. Unfreeze the account.
12. Test overdraft validation.
13. Display transaction history.
14. Test zero and negative transaction amounts.
15. Close an account with zero balance.
16. Test a deposit into the closed account.
17. Display final account details.
18. End the application.

13. PROJECT FILE STRUCTURE

The project contains the following files:

```
BankingApplication/\n    Money.java\n    AccountStatus.java\n    LedgerEntry.java\n
BankAccount.java\n  FundTransfer.java\n  Main.java
```

Each file is responsible for a particular part of the banking application.

14. COMPILATION AND EXECUTION

The application was compiled and executed using Java.

Compilation command:

```
javac *.java
```

Execution command:

```
java Main
```

The application compiled successfully without errors and displayed the banking operations and their results.

15. TEST CASES AND RESULTS

No.	Test case	Expected result	Actual result
1	Create Account A	Account created	Passed
2	Create Account B	Account created	Passed
3	Deposit ₹5,000	Balance increases	Passed
4	Withdraw ₹2,000	Balance decreases	Passed
5	Transfer ₹3,000	Transfer completed	Passed
6	Freeze account	Account frozen	Passed
7	Withdraw from frozen account	Operation rejected	Passed
8	Unfreeze account	Account active	Passed
9	Exceed overdraft limit	Operation rejected	Passed
10	Deposit zero amount	Operation rejected	Passed
11	Withdraw negative amount	Operation rejected	Passed
12	Close zero-balance account	Account closed	Passed
13	Deposit into closed account	Operation rejected	Passed

15.1 Final Account Balances

Account number	Final balance	Status
1001	₹10,000	ACTIVE
1002	₹8,000	ACTIVE
1003	₹0	CLOSED

15.2 Transaction History

Account 1001 contains the following successful transactions:

Transaction type	Amount
DEPOSIT	₹5,000
WITHDRAWAL	₹2,000
WITHDRAWAL (Transfer)	₹3,000

The final balance is calculated as:

Opening Balance + Deposits − Withdrawals

₹10,000 + ₹5,000 − ₹2,000 − ₹3,000 = ₹10,000

Therefore, the final balance matches the recorded transactions.

16. PROGRAM OUTPUT

The application executed successfully without compilation errors.

Initial account balances:

```
Account 1001: INR 10000.00\nAccount 1002: INR 5000.00
```

After deposit, withdrawal, and transfer:

```
Account 1001: INR 10000.00\nAccount 1002: INR 8000.00
```

Freeze account test:

```
Account frozen\nError: Account is not active
```

Overdraft test:

```
Error: Overdraft limit exceeded
```

Zero amount test:

```
Error: Amount must be positive
```

Negative amount test:

```
Error: Amount must be positive
```

Closed account test:

```
Account closed\nError: Account is not active
```

Final account details:

```
Account 1001: INR 10000.00 – ACTIVE\nAccount 1002: INR 8000.00 – ACTIVE\nAccount
1003: INR 0.00 – CLOSED
```

Final application message:

```
APPLICATION COMPLETED
```

```
PS C:\Users\vaibh\OneDrive\BankingApplication> javac *.java
>>
PS C:\Users\vaibh\OneDrive\BankingApplication> java Main
================================
 BANK ACCOUNT MANAGEMENT SYSTEM
================================

Initial Balances:
Account A: INR10000.00
Account B: INR5000.00

1. DEPOSIT
Balance A: INR12000.00

2. WITHDRAWAL
Balance A: INR11000.00

3. FUND TRANSFER
Transfer Successful!
Transfer ID: 4dd130a7-1457-4cdd-a25e-5490257195c5
Amount: INR5000.00
Account A: INR6000.00
Account B: INR10000.00
```

```
4. FREEZE ACCOUNT
Error: Account is frozen or closed

5. UNFREEZE ACCOUNT
Account Status: ACTIVE

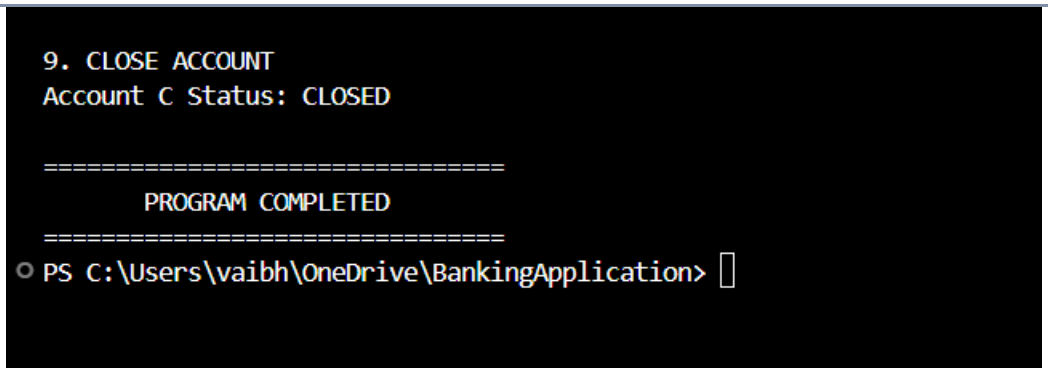
6. INVALID TRANSACTION
Error: Transaction amount must be positive

7. OVERDRAFT VALIDATION
Error: Overdraft limit exceeded

8. TRANSACTION HISTORY

Transaction History:
OPENING_BALANCE | INR10000.00 | 2026-11-12T03:19:54.399091Z | Account opened
DEPOSIT | INR2000.00 | 2026-11-12T03:19:54.409090Z | Money deposited
WITHDRAWAL | INR1000.00 | 2026-11-12T03:19:54.410090900Z | Money withdrawn
TRANSFER_DEBIT | INR5000.00 | 2026-11-12T03:19:54.411090600Z | Money transferred to another account

Transaction History:
OPENING_BALANCE | INR5000.00 | 2026-11-12T03:19:54.399091Z | Account opened
TRANSFER_CREDIT | INR5000.00 | 2026-11-12T03:19:54.411090600Z | Money received from another account
```



17. CONCLUSION

The Bank Account and Money Transfers application was successfully designed and implemented using Java and object-oriented programming concepts.

The application supports account creation, deposits, withdrawals, fund transfers, account freezing, unfreezing, closing, overdraft validation, and transaction history management.

Domain invariants and validation rules help prevent invalid transactions and maintain account correctness.

The project demonstrates important concepts such as encapsulation, abstraction, classes, objects, enums, and immutable value objects.

The application was compiled and executed successfully, and all implemented test cases produced the expected results.

This case study helped in understanding how object-oriented design principles can be applied to develop a reliable banking application.